

Digital I/O Fundamentals

Ones and zeros are voltages, and voltages have limits. This is the week where you learn what a pin can and cannot survive

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 2, printed pages 27–32.

Lab manual: Lab 03 Sections 3.1–3.2 (logic levels, seven-segment, push buttons).

1 What we are actually after this week

Last week was philosophy. This week is electricity, and it is short but unusually dense, because almost everything in it is a number you can get wrong in a way that produces smoke.

Here is the plan:

1. What a GPIO port actually is, and where it sits relative to the CPU.
2. **Inputs:** how a continuous voltage gets classified as a one or a zero, and the forbidden zone in between.
3. **Outputs:** what voltage a pin promises to produce, and the fine print attached to that promise.
4. The switch circuit, and why there is a resistor in it that seems to do nothing.
5. The LED circuit, and why there is a resistor in it that absolutely does something.
6. Scaling that up to seven LEDs at once, which is Lab 03, and where the current budget starts to bite.

Items 2, 3, and 5 are the exam material. Item 5 in particular produces a calculation question with near-certainty, so we will do it slowly and with real numbers. Item 6 is where your lab grade lives.

We are still outside the chip this week. Next week we go in and start writing to registers.

2 Deep dive 1: What is a GPIO port, exactly?

Recall from last week that an MCU is a CPU surrounded by **peripherals**, which are specialised hardware circuits that either interface with the outside world or perform some function for the MCU. The CPU executes instructions. The peripherals do everything else.

The simplest peripheral of the lot, and the one we start with, is the GPIO port.

GPIO, and its two halves

General purpose input/output (GPIO) port: a peripheral with digital input and output bits.

Input GPIO port bit: the part that lets your program *read* a single-bit signal on a pin. Connect a switch and the program can tell whether it is open or closed.

Output GPIO port bit: the part that lets your program *write* a single-bit signal to a pin, driving it to one of two voltage levels, typically 3.3 V or 0 V. Connect an LED and the program can light it.

The word "general purpose" is doing real work in that name. A UART pin can only speak UART. An ADC pin can only measure voltage. A GPIO pin is the pin that has not been assigned a specialism, so you can point it at anything: a switch, an LED, a relay, a segment of a display, the enable line of a motor driver.

Book board vs your board

The book's Nucleo-F091RC has one user switch, **B1 on PC13**, and one user LED, **LD2 on PA5**. Every example in Chapter 2 uses those two pins.

Your STM32F3 Discovery has **four** user LEDs and a user button. Your lab manual puts the user button on **PA0**. Note the polarity is opposite to the book's: the Nucleo's B1 is pulled *high* by a resistor and pulled *low* when pressed, whereas your board's user button reads *high* when pressed. That sign flip has bitten people. Check it before you write your `if`.

3 Deep dive 2: Inputs, or how a voltage becomes a bit

Here is a question worth sitting with for a second. The MCU is a digital device that thinks in ones and zeros. The outside world is analog and thinks in volts. Somewhere at the boundary, a continuous voltage has to be forced into one of two categories. Where exactly is the line?

The answer is that there is not one line. There are two, and they are defined *relative to the supply voltage* V_{DD} , not as absolute numbers.

A representative rule, and the one the book uses:

Input voltage range	Interpreted as	At $V_{DD} = 3.3\text{ V}$
0 V to $0.3 \times V_{DD}$	Logic 0	0 V to 0.99 V
$0.3 \times V_{DD}$ to $0.7 \times V_{DD}$	Undefined	0.99 V to 2.31 V
$0.7 \times V_{DD}$ to V_{DD}	Logic 1	2.31 V to 3.3 V
Above V_{DD}	Overvoltage, damage	above 3.3 V

That middle row is the interesting one. A voltage sitting at 1.6 V is not a zero and not a one. It is *undefined*, which means the pin may read it as either, may read it differently on consecutive reads, and may read it differently on the chip next to yours. That is useless to you. So the job of your external circuit is to keep the pin firmly out of that band except during the brief moment it is transitioning across it.

Beware the trap

Because the thresholds scale with V_{DD} , a circuit that works at 3.3 V can quietly stop working at 2.0 V, and vice versa. A sensor whose output tops out at 2.2 V is a solid logic one at $V_{DD} = 3.0\text{ V}$ and lands in the undefined band at $V_{DD} = 3.3\text{ V}$. Never memorise threshold voltages as absolute numbers. Memorise them as fractions of V_{DD} .

Book board vs your board

Your lab manual's Table 8.1 gives logic LOW as 0 V to about 0.8 V, and logic HIGH as 2.0 V to 3.6 V. Those are TTL-flavoured numbers and they are *more permissive* than the book's 0.3/0.7 rule at 3.3 V.

Design to the stricter one. If you keep your lows below 0.99 V and your highs above 2.31 V, you satisfy both. If you satisfy only the manual's table, you are relying on margin that the datasheet does not promise you.

3.1 Input current, or how little a pin takes

Digital inputs have **high input impedance**, meaning they draw almost nothing from whatever is driving them. The book's figure for this MCU is no more than $1\ \mu\text{A}$ per pin across the full operating temperature range, and at room temperature ($25\ ^\circ\text{C}$) it is far smaller, on the order of $25\ \text{nA}$.

Why do you care? Because it means you can drive an input pin from a very weak source, including through a large resistor, without the pin loading it down. That fact is what makes the next section work.

3.2 The 5 V question

Book board vs your board

Many STM32F303 pins are marked **FT** (five-volt tolerant) in the datasheet pin table, which means they can accept up to about 5.5 V on the input even though V_{DD} is 3.3 V. Many others are not, particularly pins with analog functions attached.

Your lab manual warns bluntly that pins are not 5 V tolerant unless explicitly stated, and the manual is right to be blunt, because the consequence of getting it wrong is a dead pin. The correct procedure is to open the pinout table in the datasheet and look for FT next to the specific pin you are about to abuse. Not the family. The pin.

4 Deep dive 3: Outputs, and the fine print on the promise

Now the other direction. Your program writes a 1 to an output pin. What voltage actually appears?

Two symbols to learn:

V_{OH} and V_{OL}

V_{OH} (*output high*) is the voltage the pin produces for a logic one. Specified as somewhere between $V_{DD} - 0.4\ \text{V}$ and V_{DD} .

V_{OL} (*output low*) is the voltage the pin produces for a logic zero. Specified as somewhere between 0 V and 0.4 V.

Note that neither is exactly at the rail. There is always a small drop, because there is a transistor in the way.

And here is the fine print, which is the actual content of this section: **that specification only holds if you draw a limited amount of current from the pin.**

Think about why. The output stage is a transistor acting as a switch, and a real transistor has resistance when it is on. Current through resistance produces a voltage drop. So the more current you pull out of a pin driving high, the further the output voltage sags away from V_{DD} . Pull enough and the "logic one" you are producing drifts down into the undefined band, and whatever is reading it stops believing you. Pull much more than that and the output transistor overheats and dies, permanently, taking that pin with it.

Book board vs your board

	Book (STM32F091RC)	Yours (STM32F303VCT6)
Max current per pin, I_{OH} / I_{OL}	25 mA	25 mA (absolute max)
Current at which $V_{OL} \leq 0.4$ V is guaranteed	—	8 mA
Total current into V_{DD} / out of V_{SS}	—	150 mA
Operating voltage	2.0–3.6 V	2.0–3.6 V

The 25 mA figure is an *absolute maximum*, the number past which the manufacturer stops promising the pin survives. It is not a design target. The 8 mA figure is where the datasheet actually guarantees the output voltage specification. Design at 8 mA or below and you are inside the guarantee. Design at 20 mA and you are relying on a "relaxed" specification with a much worse V_{OL} . Design at 25 mA and you are gambling.

The 150 mA total is the one people forget, and it is the one that matters for Lab 03. More on that shortly.

One more dependency worth knowing, because it explains a class of bug that looks like magic. **Drive strength falls as V_{DD} falls.** At lower supply voltage the output transistors have higher resistance, so the same pin can supply less current before the voltage spec breaks. The book's illustration: dropping V_{DD} from 3.3 V to 2.0 V might cut the usable I_{OH} and I_{OL} from 5 mA down to 1.5 mA. If you are running from a battery that sags as it discharges, your LEDs will visibly dim before anything else goes wrong, and that is the reason why.

5 Deep dive 4: The switch circuit, and its useless-looking resistor

Right. Circuit one, the input side. A switch and a **pull-up resistor** connected to the input pin.

The wiring: the pin connects through a resistor up to V_{DD} , and also connects through the switch down to ground.

- **Switch open.** No path to ground. The only path is up through the resistor to V_{DD} . Recall from Section 2.1 that the pin draws essentially no current, on the order of nanoamps. So there is essentially no current through the resistor, so by Ohm's law essentially no voltage drop across it, so the pin sits at V_{DD} . **Reads logic one.**
- **Switch closed.** Now there is a direct short from the pin to ground, which wins. **Reads logic zero.**

So why is the resistor there at all? Ask the opposite question: what if you removed it and just wired the switch from pin to ground?

Then with the switch open, the pin connects to nothing whatsoever. It is **floating**, and a floating high-impedance input does not read zero. It picks up whatever stray charge and electromagnetic noise happens to be around, drifts wherever it likes, and typically lands somewhere in that undefined band we discussed. Your button will appear to press itself when you wave your hand near it. This is one of the great rites of passage in embedded work.

Pull-up and pull-down

A resistor that connects a signal line to V_{DD} (pull-up) or to ground (pull-down), so the line has a *defined* default level when nothing else is actively driving it. Sized large, typically 10 k Ω , because it only has to overcome leakage, and because when the switch closes you want the wasted current to be small. At 3.3 V through 10 k Ω that is $I = V/R = 3.3/10000 = 0.33$

mA. Negligible.

Book board vs your board

The book's Nucleo uses a **pull-up**, so the pin reads 1 when idle and 0 when pressed. Active low.

Your lab manual specifies a **pull-down** for external buttons: one leg to the GPIO pin, the other leg to 3 V, with a 10 k Ω resistor from the pin to ground. So the pin reads 0 when idle and 1 when pressed. Active high. The manual's own example reflects this:

```
1 | if (HAL_GPIO_ReadPin(GPIOD, GPIO_PIN_7)) {
2 |     // runs when the button IS pressed
3 | }
```

Neither convention is more correct. But you must know which one your circuit uses, because getting it backwards produces a program that works perfectly and does everything at exactly the wrong moment.

Beware the trap

You do not always need an external resistor. Every STM32 GPIO pin has an internal pull-up and an internal pull-down available in software, selectable in CubeMX or through the GPIO_InitTypeDef structure. They are weak, in the tens of k Ω , which is fine for a button. The trap is forgetting they exist and soldering a resistor you did not need, or worse, enabling an internal pull-up while also fitting an external pull-down and then wondering why the pin sits at a bewildering 1.4 V. Pick one.

6 Deep dive 5: The LED circuit, and its very necessary resistor

Circuit two, the output side, and this is the one that generates exam questions.

An LED is a diode, so it is polarised. The **anode** is the positive terminal, drawn as the base of the triangle. The **cathode** is the negative terminal, drawn as the bar. Current flows anode to cathode and only anode to cathode.

In our circuit the anode connects to the MCU output pin through a resistor, and the cathode goes to ground. Write a 1 to the pin and current flows and the LED lights.

Now, the crucial physics. **An LED's brightness is roughly proportional to the current through it, and its current is a violently nonlinear function of the voltage across it.** Nearly zero current below a threshold, then rising extremely steeply above it. The book gives real numbers for a red LED:

Voltage across LED	Result
1.6 V	May not light at all
2.0 V	Bright, about 20 mA
2.3 V	Brighter, current shoots up to about 43 mA
Higher still	Overheats and fails

Look at that middle jump. A 0.3 V increase, about 15%, more than doubles the current. That is what "exponential" means in practice, and it is why you cannot control an LED by controlling its voltage. The knob is far too sensitive.

And here is the collision. A GPIO output does not offer fine voltage control anyway. It offers exactly two levels: near ground, and near V_{DD} . Near ground is fine, the LED stays dark. Near

V_{DD} , say 3.3 V, is far past the point where the LED's current runs away, and it will burn out the LED, and quite possibly the pin driving it.

So we do not control the voltage. We insert a resistor and control the **current**.

6.1 Sizing the current-limiting resistor

The circuit is a series loop: supply, resistor, LED, ground. The voltage across the LED settles at roughly its **forward voltage** V_F , a property of the LED itself, and whatever is left over lands across the resistor. So:

$$R = \frac{V_{DD} - V_F}{I_{LED}}$$

Three steps, in this order:

1. Pick I_{LED} that is safe for *both* the LED and the MCU pin. The pin's limit is the binding one, usually.
2. Look up V_F for your specific LED. It depends on the colour, because it depends on the semiconductor.
3. Divide.

Worked example, straight from the book. Take $I_{LED} = 4$ mA and $V_{DD} = 3.0$ V.

Red LED, $V_F = 1.8$ V:

$$R = \frac{3.0 - 1.8}{0.004} = \frac{1.2}{0.004} = 300 \, \Omega$$

Blue LED, $V_F = 2.7$ V:

$$R = \frac{3.0 - 2.7}{0.004} = \frac{0.3}{0.004} = 75 \, \Omega$$

Notice how different those are, and notice *why*. The blue LED has a much higher forward voltage, so there is far less voltage left over for the resistor, so the resistor must be far smaller to let the same current through. Swapping a red LED for a blue one without changing the resistor will make it dim or dark, and people spend real time debugging that.

Beware the trap

Forward voltage varies by colour and by part, roughly 1.8–2.2 V for red, 2.0–2.4 V for yellow and green, 2.7–3.4 V for blue and white. That last range is a problem: a white LED with $V_F = 3.4$ V simply cannot be driven from a 3.3 V rail at all, because there is no headroom left. No resistor value fixes that. You need a higher supply or a boost driver. If your white LED is stubbornly dim, this is usually why, and no amount of code will help.

7 Deep dive 6: Seven LEDs at once, which is Lab 03

Now let's scale it, because a seven-segment display is not one LED. It is seven, plus a decimal point, in one package arranged in a figure eight, labelled a through g plus DP.

They come in two flavours, and the distinction controls your entire code logic:

Type	Common terminal	To light a segment
Common cathode	All cathodes tied together, to GND	Drive the segment pin HIGH
Common anode	All anodes tied together, to 3 V	Drive the segment pin LOW

Your lab manual's lookup table in Section 3.9 is for a **common anode** display, which is why it looks inverted at first glance. Digit 8, which lights all seven segments, is 0b0000000, hex 0x00. Every bit zero, because zero means on. Digit 1, which lights only segments b and c, is 0b1111001, hex 0x79. Bits are in order gfedcba, so bit 0 is segment a.

In the lab

Step one of Lab 03, before you write any code: determine which type you have. The manual says to check the datasheet or use a multimeter. The multimeter method is faster. Put it in diode-test mode, hold the black probe on the common pin, and touch red to a segment pin. If a segment lights, the common is the cathode. If nothing lights, swap the probes, and if it lights now, the common is the anode.

Get this wrong and your display shows the photographic negative of every digit, which is at least an entertaining bug.

7.1 The current budget, which is the real lesson

Here is where this week's electrical content stops being theory. Digit 8 lights all seven segments simultaneously. So:

$$I_{\text{total}} = 7 \times I_{\text{segment}}$$

Run the numbers against your F303's limits:

Per segment	Total for digit 8	Per-pin limit (25 mA)	Verdict
4 mA	28 mA	fine	Comfortable, inside the 8 mA guarantee
8 mA	56 mA	fine	Still inside the guarantee
20 mA	140 mA	fine per pin	Within 93% of the 150 mA total budget
25 mA	175 mA	at the limit	Exceeds the 150 mA total. Do not.

Read the last two rows carefully, because they contain the point. At 20 mA per segment, *every individual pin is comfortably legal*, and the chip is still in trouble, because the 150 mA ceiling applies to the total current through the V_{DD} and V_{SS} pins for the whole device. Seven legal pins can add up to one illegal chip.

Beware the trap

Your lab manual's Lab 03 wiring instructions say to connect each segment to a separate GPIO pin and the common terminal to GND or 3 V. It does not mention series resistors anywhere.

Fit them anyway, one per segment, sized as in Section 5.1. Without a current-limiting resistor you are connecting a 3.3 V source directly across an LED with a ≈ 2 V forward voltage, and the current is then limited only by the pin's own output resistance, which is not a design, it is an accident. Some display modules have resistors built onto the module, in which case you are fine, but you check that by inspecting the module rather than by assuming.

This is also the answer to give in your viva if you are asked why your circuit has components the manual did not list.

7.2 Writing all seven bits at once

One efficiency note that connects forward to next week. The obvious implementation is seven separate calls:

Listing 1: The obvious way. Correct, and wasteful.

```

1 void display_number(uint8_t num) {
2     uint8_t segment_map[16] = { 0x40, 0x79, 0x24, 0x30, 0x19, 0x12,
3                                   0x02, 0x78, 0x00, 0x10, 0x08, 0x03,
4                                   0x46, 0x21, 0x06, 0x0E };
5     uint8_t pattern = segment_map[num];
6     for (int i = 0; i < 7; i++) {
7         HAL_GPIO_WritePin(GPIOx, GPIO_PIN_y[i], (pattern >> i) & 1);
8     }
9 }

```

That is seven function calls, seven register accesses, and seven separate moments in time. The segments do not change together, they change in a ripple. For a slow display nobody notices, but the principle matters.

If your seven segment pins happen to sit on consecutive bits of the same port, you can set all of them in a single write, because the port's output data register is one 16-bit register covering all sixteen pins:

Listing 2: The port-wide way. One write, all seven segments change simultaneously.

```

1 | GPIOA->ODR = (GPIOA->ODR & ~0x7F) | pattern;

```

Take that line apart, because it is the pattern you will use constantly from next week onwards. `~0x7F` is `0xFF80`, a mask with the bottom seven bits cleared. ANDing the current ODR with it wipes the seven segment bits while preserving the other nine pins on the port. ORing in `pattern` then sets the segment bits to what you want. Read, modify, write.

Read-modify-write

The three-step move for changing some bits of a register without disturbing the rest: read the current value, clear the bits you intend to change with an AND against an inverted mask, set them to their new values with an OR, write the result back. Writing the register directly instead would zero every other pin on the port, and one of those pins is usually something you cared about.

That is the whole idea behind port-based organisation, incidentally. The manual explains it well: pins are grouped into ports of sixteen, each port has its own block of memory-mapped registers, and because the data register is a full sixteen bits wide, you can operate on the whole port in one instruction rather than pin by pin.

And that is a wrap for the outside of the chip. Everything from here on is inside.

8 Tying it back

We started with a question that sounds almost too simple to ask: how does a voltage become a bit?

The answer turned out to have a shape. On the input side, thresholds defined as fractions of V_{DD} , with a forbidden band in between that your circuit must not linger in, which is why a floating pin is a bug and a pull-up resistor is the fix. On the output side, a promised voltage that is only promised if you keep the current below a stated limit, which is why an LED needs a series resistor, and why the value of that resistor depends on the LED's colour, and why seven of them at once runs into a chip-wide current budget that no single pin ever violates.

Every one of those is an electrical constraint, and the course syllabus lists exactly that as this week's goal. Next week we stop looking at the pin and start looking at the registers behind it: how the CPU reaches a pin at all, why you have to switch on a clock before the peripheral will answer, and what CMSIS is doing between your C code and the silicon.

Cheat sheet: Week 02

Input thresholds, as fractions of V_{DD}

Logic 0: 0 to $0.3V_{DD}$ · Undefined: $0.3V_{DD}$ to $0.7V_{DD}$ · Logic 1: $0.7V_{DD}$ to V_{DD}

At $V_{DD} = 3.3\text{ V}$: **0 – 0.99 V** is zero, **0.99 – 2.31 V** is undefined, **2.31 – 3.3 V** is one.

Input current: $\leq 1\text{ }\mu\text{A}$ over full temperature, $\approx 25\text{ nA}$ at 25°C . High impedance.

Output specifications

V_{OH} between $V_{DD} - 0.4\text{ V}$ and V_{DD} . V_{OL} between 0 V and 0.4 V .

Only valid if current is limited. More current $\Rightarrow$ bigger drop across the output transistor $\Rightarrow$ output sags away from the rail $\Rightarrow$ eventually the transistor fails.

Drive strength *falls* as V_{DD} falls. $3.3\text{ V} \rightarrow 2.0\text{ V}$ can cut usable current from 5 mA to 1.5 mA .

STM32F303VCT6 current limits, memorise these three

25 mA per I/O pin, absolute maximum, not a design target.

8 mA per pin is where $V_{OL} \leq 0.4\text{ V}$ is actually guaranteed. Design here.

150 mA total into V_{DD} / out of V_{SS} for the whole chip. Seven legal pins can break this.

Operating range $2.0\text{--}3.6\text{ V}$. 5 V tolerance only on pins marked **FT** in the datasheet pin table.

The two circuits

Switch: pull-up (or pull-down) resistor, $\approx 10\text{ k}\Omega$, gives the pin a defined level when the switch is open. Without it the pin *floats* and reads garbage. Internal pulls are available in software, use one or the other, never both.

LED: anode = positive = triangle base. Cathode = negative = bar. Brightness $\propto$ current. Current is exponential in voltage, so control current, not voltage.

$$R = \frac{V_{DD} - V_F}{I_{LED}}$$

Worked: $V_{DD} = 3.0\text{ V}$, $I_{LED} = 4\text{ mA}$. Red ($V_F = 1.8\text{ V}$) $\rightarrow 300\text{ }\Omega$. Blue ($V_F = 2.7\text{ V}$) $\rightarrow 75\text{ }\Omega$.

Typical V_F : red $1.8\text{--}2.2\text{ V}$, green/yellow $2.0\text{--}2.4\text{ V}$, blue/white $2.7\text{--}3.4\text{ V}$.

Seven-segment, for Lab 03

Common cathode: common to GND, segment HIGH turns it on.

Common anode: common to 3 V , segment LOW turns it on. *The manual's lookup table is common anode.*

Bit order gfedcba, bit 0 = segment a. Digit 8 = $0x00$ (all on), digit 1 = $0x79$.

Digit 8 lights 7 segments at once: $I_{total} = 7 \times I_{seg}$. At 20 mA each that is 140 mA of a 150 mA budget.

Fit one series resistor per segment. The manual does not mention them. Fit them anyway.

Read-modify-write: `GPIOA->ODR = (GPIOA->ODR & ~0x7F) | pattern;`

Likely exam prompts from this chapter

- Given V_{DD} , V_F , and a target current, compute the series resistor. *Near certain.*
- Why does an input pin need a pull-up or pull-down? Answer: floating pin, undefined band.

- Why can a GPIO pin not drive an LED directly? Answer: two-level output, exponential I-V curve, no current limiting.
- What happens to V_{OH} as output current rises, and why?
- Distinguish per-pin current limits from total device current limits.